Azure Assignment - PetfAIndr

To-Do´s

Overview

GENERAL INFORMATION 2

ChatGPT 2

Azure Region 2

Azure Subscription 2

Recommended timing 3

EXPECTED DELIVERIES 4

Pricing overview 4

Architectural diagramm and process worklow 4

The running App in Azure 4

PETFAINDR – THE DETAILS 7

Context 7

The Developer situation 8

The Petfaindr Solution 8

Building and pushing the container-Images 10

The whole solution up in the Cloud 11

General Information

CHATGPT

We expect that you are familiar with the usage of ChatGPT or Microsoft Copilot. Therefore some basic functions as well as some advanced codes have not been specified, since we assume you will use this GenAI Tool to achieve the target of the assignment!

AZURE REGION

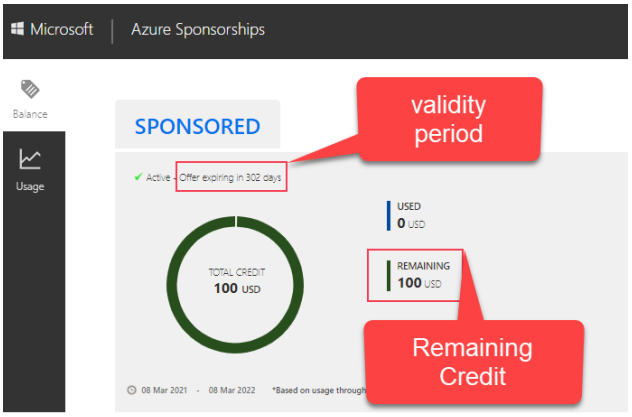
Due to the very likelihood of deployment failures based on the restrictions of the FHNW Azure Subscription, it is highly recommended to create all Azure resources in the Azure Region **Sweden Central** (if possible – CosmosDB might be an exception).

AZURE SUBSCRIPTION

Please ensure that one team member which hasn´t created an Azure Subscription for the Hands-Ons during the lectures creates a new Azure Subscription to host the Assignment environment. If you leverage the already created Azure Subscription it might led to the problem that you will run out of Azure credits before you have to deliver the Assignment!

How to get a new Azure Subscription and grant the necessary permission, is described in the documentation “Get-An-Azure-Subscription” which you can find your TEAMs File section.

Your FHNW provided subscription is worth $100 CHF and will work for 365 days.
Ensure that you will have sufficient Azure Credits until the exam validation in June!
You can always validate this, by visiting https://www.microsoftazuresponsorships.com/Balance (use your FHNW Account to login) and check for the validity period and the remaining credits. Alternatively you might use the Cost Analysis feature for your Subscription in the Azure Portal.



You might want to make use of the AKS Cluster stop-feature described here to save your valuable Azure Credits!

RECOMMENDED TIMING

1.) 10h – Technical implementation

Expect that your team will approximately require 10h in total to finish the technical tasks. This of course totally varies based on the technical knowledge your team has.

2.) 5h – Cost, Architecture and process workflow

Reserve approximately 5h in your last two weeks of the assignment to do task 1 and 2. By then you really now which resource are required for a running solution.

General HINT: Start as early as possible with the technical implementation, so that you can make the most out of the coaching calls happening every week before the final delivery.

Expected deliveries

PRICING OVERVIEW

A pricing calculation of the final architecture for all used Azure resources. It needs to take into account, that failover to another region should be possible to run at least in a Disaster Recovery setup (smallest size possible).
Therefore please provide the figures for the two Azure Regions
- Sweden Central
- Switzerland North

Makes use of the Azure Pricing Calculator to get the pricing estimates. Leverage the benefits of the so called 3 year reserved-instances, where applicable.
Please make use of a sufficient node size for the Kubernetes Cluster (that has buffer for scaling and installing updates on the fly) – not the B2s series!

ARCHITECTURAL DIAGRAM AND PROCESS WORKLOW

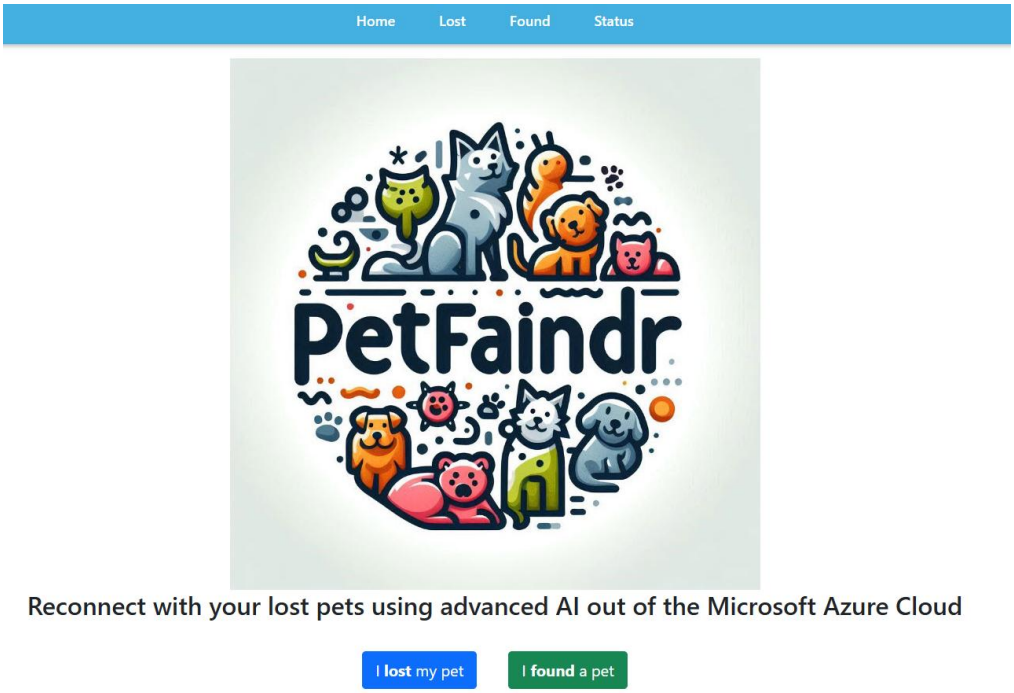
Please provide a diagram of the final architecture of the solution hosted in Azure. Please **do not spend too much time** on selecting the right tool for visualization. Instead, focus on the target to deliver an architectural overview which you can explain.

In addition please provide the action blocks of the overall setup process (e.g. Prep Developer-Workspace, Create Azure Resources, Deploy to K8s, etc.) which have led to the target solution running in Azure.

Hint: You can choose e.g. to use Paint, MS Whiteboard, OneNote, Visio, Word, PowerPoint – all are legitimate options and will have zero impact on the grading. Also think of the baseline you can get from the Azure Resource Visualizer as a starting point.

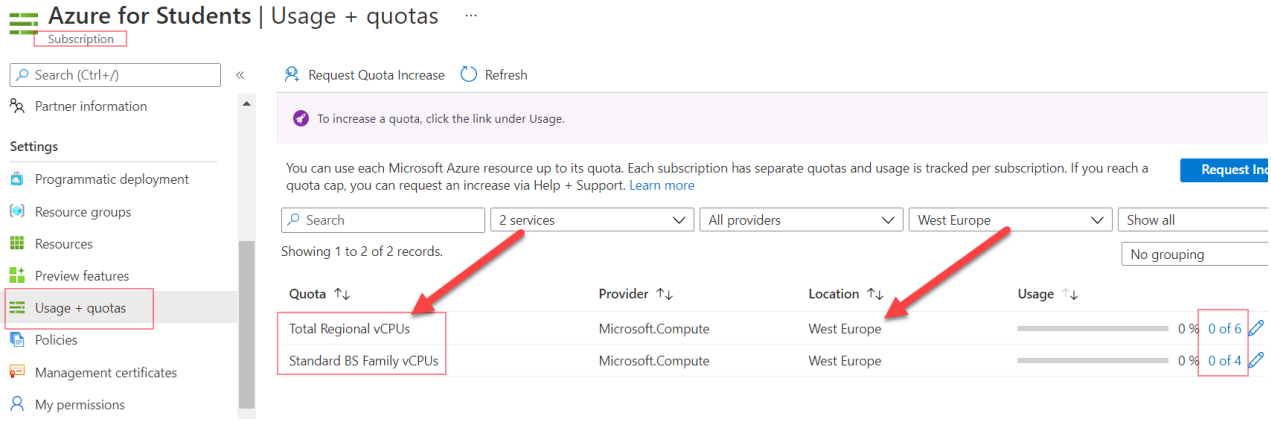
THE RUNNING APP IN AZURE

- You should be able to present the running PetfAIndr solution live on Azure and navigate through all three sites, especially the status page.
- Additionally you need to have one of the three provided training Dogs marked as “new” in the status overview, so that the lecturer can validate the AI endpoint for this specific Dog with an unseen image for the AI. This implies you have to provide the full URL of the app to the lecturer for validation purpose.



-

Due to FHNW Subscription Limits, make sure to use only two VMs for the Kubernetes cluster based on the type “Standard_B2s”. Otherwise your cluster will fail to be provisioned, since you don´t have quota for more.
This sizing of the K8s Nodes is NOT to be used for the pricing calculation!
Since the current quota for vCPUs is set to 4 per Region, you might also think about where (Azure Region) you would like to place your development solution/VM, if you are going for this approach. You can validate how much quota is left for vCPUs by checking the “Usage + quotas” details of your Subscription and look for “Total Regional vCPUs”.



ADDITIONAL TASKS AND QUESTIONS

- Describe your solution which solves “the Developers situation” and be able to explain why you have chosen it – ChatGPT can´t do that for you in the exam!

- Give at least three ideas how the PetfAIndr suite (GUI enhancements are not an option!) or the deployment could be improved.

Hints:

- Use the Azure Architecture Center to get an idea how an Architectural Diagram could look like. You might want to leverage the free Azure Visio Stencils.

All of the deliveries (combined in ONE PDF/PPTX) mentioned above need to be stored in the Teams Channel folder for the Assignment until
Monday 1st June 2026 23:59 o´clock!

PetfAIndr – The details

CONTEXT

Introducing **PetfAIndr**, your innovative startup set to transform the way we all find lost pets using the power of AI and community collaboration!
Your team of exceptional entrepreneurs just recently attended Europe's largest university pitching contest, **TalentPitch**, in Zurich and you have been truly inspired by the heartwarming story of **PawsPulse**. BUT - the visionary team behind PetfAIndr is determined to make pet recovery faster, more efficient, and of course more sustainable by leveraging cutting edge AI empowered technology originating out of the Microsoft Azure Cloud.

Leveraging the **Microsoft Azure AI Stack**, PetfAIndr is an AI-enabled app designed to make it easier to locate lost pets. Pet owners can report their missing pets, and the community can upload pictures of pets they believe might be lost. The AI then analyses these images to determine if the pet has been registered as missing, streamlining the search process and increasing the chances of a happy reunion!

Join us on this thrilling journey as **PetfAIndr** revolutionizes pet recovery, making it smarter, more responsive, and community-driven.

>>> Welcome to the future of finding lost pets! <<<

Technical insights
You would like to develop the product in a way that you could easily distribute and scale it worldwide and also collaborate with developers which you might engage all over the planet. To achieve this goal, the agile development team decided to leverage the capabilities of an microservice based architecture for the application based on a Blazor Web frontend and a Python written backend. Both make use of DAPR enabled containers which run on Kubernetes while using an Azure Cosmos DB and the latest Azure Custom Vision AI Services in combination with Azure Storage. They communicate asynchronously via Azure Service Bus.

TIP: **Read** the instrcutions carefully and until the end – in the end, that will save you time, pain and headaches!
VERY STRONG RECOMMENDATION – Use the Infrastructure-as-Code (IaC) paradigm wherever possible. This means that you should always prefer the Powershell or Bash Code statement to achieve a task in Azure, instead of leveraging the Azure Portal - e.g. "`az storage account create`" to create an Azure storage account, or use the deployment script which are automatically provided when you create a new service using the Azure Portal.

THE DEVELOPER SITUATION

Think of a solid solution that is not expensive and could easily be adjusted to the needs of the developers if the need more capacity. Also ensure that you do not have latency issues when your developer is located outside of Switzerland. Don´t make him dependent on a physical notebook which is governed by the startup.
You can use this solution to work on the implementation of the application setup in Azure.

Given the nature of this project (and the current restrictions by Microsoft that do not offer ACR Tasks for Free-Subscriptions), you have to use a Windows OS based machine which has at least docker installed on it. To connect to the Windows VM hosted on Azure you can use the mstsc-Tool (Windows User)/Windows App (MAC User). Deploy the resource in an Azure Region which is not Sweden Central to avoid resource availability issues for your Kubernetes environment.

THE PETFAINDR SOLUTION

The whole PetfAIndr suite has been developed in a very lean way. For the running app, only two container images are needed on an Azure Kubernetes Cluster.

- There is a Blazor based frontend which is containerized in the “frontend” container and
- a Python based backend which handles the business logic and the AI-Modell triggering called “backend”.

Those two containers should later on run on the Azure Kubernetes Cluster that has the DAPR (version 1.16) functionality enabled. DAPR needs to be manually activated in advance.

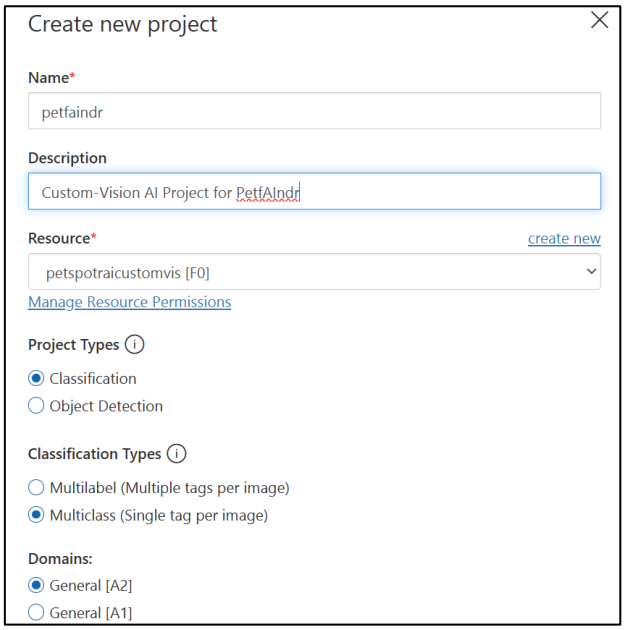
To achieve the best possible user experience, the frontend container should be accessible over a public URL provided by Azure using a public IP Address and a Kubernetes Ingress Controller.

Advice: For the easiest usage of the whole application and their components, refer to the naming of the components in the file “aks-config.bicep”.

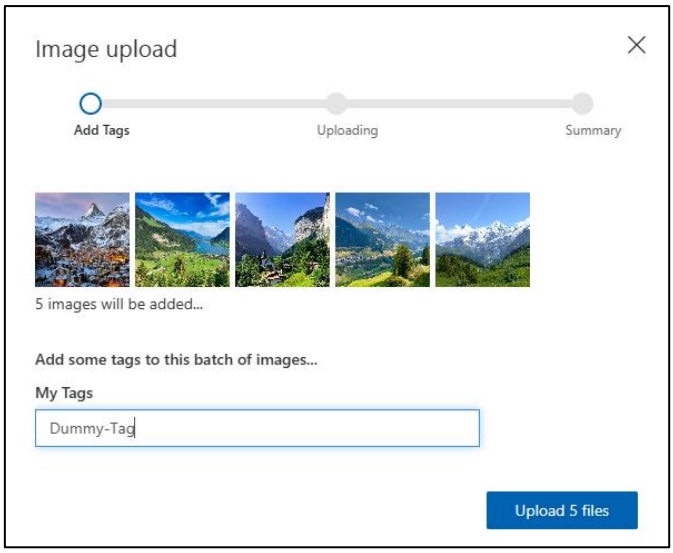
Besides this, the PetfAIndr suite assumes the existence of a

- CosmosDB
  - Type: NoSQL
  - DB id: petfaindr
  - Container id: pets
  - Partition Key: /partitionKey
  - Throughput: Manual/400 RU/s
- Storage Account
  - BLOB Anonymous access enabled
  - A Container called “images” with anonymous access enabled

- Azure Service Bus
  - Two topics (foundpet / lostpet) (Name them exactly like this, pay attention to the case-sensitivity!)
  - a shared access policy called “Dapr” with Manage Permissions
- An Azure Custom vision service
  - The Service should provide a Training AND Prediction API while using the Free-F0 type (SKU)
  - An Azure Custom Vision project through the Azure custom Vision Portal with the following settings (once it is created you´ll get the project-id from here):



  - To ensure that the app can run, the images in the “Dummy-Tag-Images” Folder need to be uploaded in your AI project through the Azure custom Vision Portal and tagged as “Dummy-Tag”.



BUILDING AND PUSHING THE CONTAINER-IMAGES

To build the mentioned container images above use the source code which has been provided in the PetfAIndr.zip File in your Assignment-Teams Channel Folder. There you will find the necessary Dockerfiles to build the container images on a **Windows machine** and push them to the Azure container registry.

Make sure you have an Azure Container Registry (ACR) setup in advance and check the documentation Build and push an image from Dockerfile to get the detailed commands. Remember that you have to use the Windows machine (e.g. VM) with Docker installed on it to build and push the container images to the ACR.

Further helpful references

Docker

- Getting Started with Docker
- Docker Networking
- Dockerfile reference
- Docker CLI reference

Azure

- Azure Developer Enabelement Options
- Azure CLI reference in the CloudShel
- Azure CLI for Linux
- Azure Container Registry
- Azure Quickstart to deploy a Kubernetes Cluster

THE WHOLE SOLUTION UP IN THE CLOUD

At this point, you've built the images for the components of your application and made those images available in your private ACR. Now, your team's goal is to deploy your application to an Azure Kubernetes Service (AKS) cluster in your Azure subscription and have it connected to your private ACR.
You might want to STOP your cluster when you do not work on it to save costs (ensure you have the IaC Commands saved – BEFORE you delete). Check this article to read how you can achieve this.

For pre-calculating the final production environment (which you are not going to build) , you should consider that you will have both Microservices up and running with at least 2 pods of each Microservice. In general one Microservice should have 2 GB RAM and access to at least one CPU. Also take into account to have buffer for upscaling some nodes on demand and that you have sufficient resources to upgrade the whole Kubernetes cluster. Your sizing needs also to take into account the additional overhead that the DAPR framework adds to the load of the cluster.

The aks-config.bicep file

Secrets! They are important in order to achieve different levels of security! Therefore the PetfAIndr suite makes use of them as well. Technically this will be achieved by using - Kubernetes Secrets – in the Kubernetes Cluster.
BEFORE you deploy the application containers to the Kubernetes Cluster, ensure that you have CAREFULLY inserted all necessary variables in the file (aks-config.bicep) and that your Azure resource like the storage account or the Cosmos DB comply with the naming conventions used in the file. Otherwise provide the different names of the resources as a parameter value when you run the file.
You can run this bicep file in the context of your Azure Subscription, to ensure the secrets are set in the Kubernetes Cluster.

The AKS Cluster

Check the documentation link called - How to setup an Azure Kubernetes Cluster – for further information on how to setup an AKS Cluster (or review the recording of the 3rd Hands-On Session after Darth Vader riding the white cat) and deploy an application to it. Ensure your "AKS Create command" has the following parameters in it, to enable the connection to your Azure Container Registry (ACR): `--attach-acr <yourACRName>`

Due to the current setup/limitation of the FHNW Subscriptions, you might experience the following error:

```
Operation failed with status: 'Bad Request'. Details: Provisioning of resource(s) for container service aks-[redacted] in resour
e group rg-[redacted] failed. Message: Operation could not be completed as it results in exceeding approved Total Regional Cores
quota. Additional details - Deployment Model: Resource Manager, Location: westeurope, Current Limit: 6, Current Usage: 1, Additiona
 Required: 6, (Minimum) New Limit Required: 7. Submit a request for Quota increase at https://aka.ms/ProdportalCRP/?#create/Microso
```

This error most likely occurred, since you have your resources not deployed in Sweden Central, or you have chosen a K8s Node VM Type which consumes more than the allowed 4 vCPUs per Region (most likely you have **NOT** chosen the defined VM Type "Standard_B2s"). If you would like to validate in advance how much resources are left in which Azure region, you could easily do this by visiting the "Usage+quota" Settings in your Azure Subscription in the Azure Portal.

Both applications in the respective containers assume some values being configured as K8s secrets. Therefore ensure that you have updated all values in the aks-config.bicep file that are marked accordingly (like "your key …"). In addition, ensure that you use the correct naming for the components like the Kubernetes Cluster, the Cosmos-DB and so on – otherwise you need to call the script with the name you have choosen for the resources through the respective parameters.

In order to create the running container for the frontend and the backend, you can make use of the bicep template "app.bicep" in the provided zip-file.
BEFORE you execute it – ensure that you have named your images in the ACR like they are referenced in the files ./app/backend.bicep and ./app/frontend.bicep (check for "image").

Otherwise you most probably will run into a "ImagePullBackOff" Error while the AKS Clusters tries to pull the container-image out of your container registry.

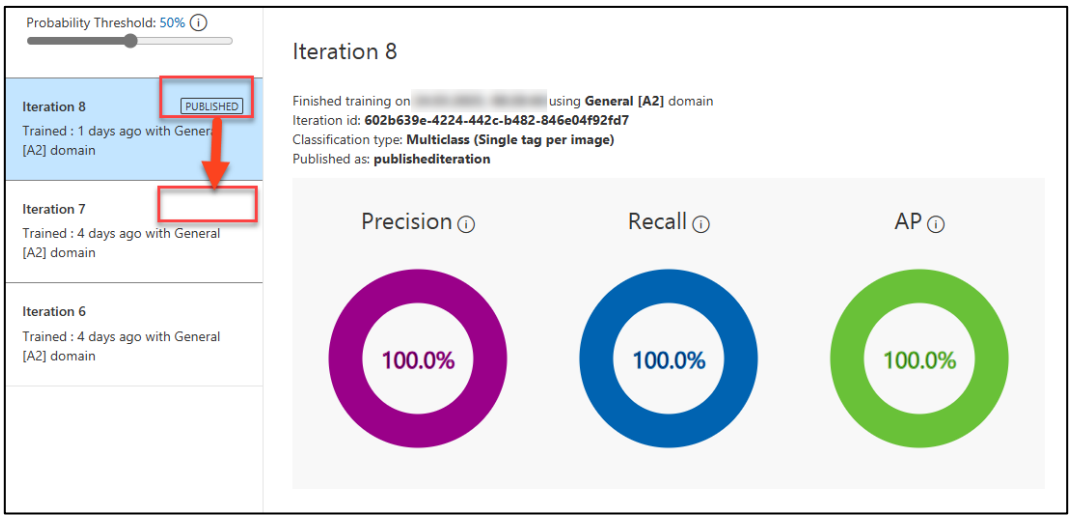
Lastly, you need to ensure that the enabled DAPR service on your AKS Cluster is properly configured. Therefore you should apply the three YAML configuration files in the DAPR folder against your Kubernetes Cluster, so that the DAPR components "pets", "images" and "pubsub" are registered in the Cluster.
This needs to be done !!! BEFORE !!! your deploy the application containers!

Training of the underlying AI Modell

Once you have the PetfAIndr suite up and running in the cloud and reachable over the configured ingress controller, you can train the model with the provided Dog images. Those are stored within the folder pet-images in the zip-file.
Please respect the following guidelines:

1.) Only ONE Dog per Submission (while one submission should contain the 5 images of one Dog)
2.) You will have to wait approximately 12 Minutes until the AI Model has been trained with the new imagery once you hit the “Submit” button, before you can validate the lost pet through the “found-pet” Site

3.) Use the status site to get an overview of already registered Dogs
4.) You can use your own images of pets, if you like – but only the provided Dog images have been tested with the AI Custom Vision service to produce accurate findings.
5.) To check the current status of a published Model-“Iteration” you can visit the “Performance” section in the Custom Vision AI Portal. There you can see the current status of the training, as well as the current published iteration. ONLY if there is a published iteration, the “found pet” site can work properly.



6.) As of now – the developers decided to not provide any kind of insights into whats happening behind the scenes – meaning what the backend container is really doing. For this, you might consider using the kubectl command that shows you logs from a specific pod in order to see the log messages from the backend container. Those should provide you more insights into whats happening behind the scenes.

References - Kubernetes:

- Kubernetes core concepts
- Kubectl overview
- How to setup an Azure Kubernetes Cluster
- How to get the Azure Kubernetes Credentials to login with kubectl
- How to start and stop (Pause) a cluster
- Kubernetes secrets
- Kubernetes services

Azure Kubernetes Service (AKS)

- Deploy an AKS cluster using Azure CLI
- Azure CLI: az aks create

Azure

- Azure CLI reference
- Resource naming conventions